# SIMATS ENGINEERING

## ASSESSMENT TOOL 2: Industry Problem Solving Task

**COURSE NAME:** Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1507

---

## COURSE OUTCOME COVERED

**CO4:** *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 40**
Question Count: 5

Duration :60 Minutes

---

## Code of Conduct

I Athavan K(192524036)that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: Athavan K

## Assessment Tasks Industry Problem Solving Task

1. **Retail Data Optimization Problem -** A retail chain operates both physical stores and an online platform. It generates large volumes of sales, inventory, and customer behavior data. The company wants real-time inventory tracking and demand forecasting. It also aims to deliver personalized marketing recommendations to customers. Design a big data architecture to support these requirements. Explain the tools, technologies, and analytics models you would use.

2. **Social Media Fraud Detection Problem -** A social media platform processes billions of user interactions daily. These include posts, comments, likes, and shares. The platform wants to detect fake accounts and misinformation quickly. Real-time or near real-time processing is required. Design a scalable big data solution for this use case. Explain the role of machine learning and stream processing frameworks.

3. **Government Data Platform Problem** A government agency is building a national data platform. It combines census, economic, and geospatial datasets. Different departments need access to shared and integrated data. The system must ensure interoperability across multiple agencies. Strict privacy and security requirements must be followed. Propose a governance and data management strategy.

4. **Banking Fraud Detection Problem**

A bank processes millions of financial transactions every minute. It needs to detect fraudulent activities in real time. The system must analyze historical and streaming transaction data. Low latency and high accuracy are critical requirements. Design a big data pipeline for fraud detection. Explain the technologies and algorithms you would implement.

5. **Agriculture Smart Farming Problem**

A smart farming system collects soil, weather, and crop data. Sensors and drones provide real-time field information. Farmers want insights to improve yield and resource usage. Data is generated from multiple distributed sources. Design a big data solution for precision agriculture. Explain how analytics can support decision-making.

## Industry Problem Solving Rubric

| Criteria | Weightage | Level 4 - Exemplary | Level 3 - Proficient | Level 2 - Developing | Level 1 - Beginning |
|---|---|---|---|---|---|
| Understanding of Industry Context | 20% | Demonstrates strong understanding of real-world problem context | Good understanding | Basic understanding | Weak understanding |
| Application of Big Data Concepts | 25% | Applies highly relevant big data ideas and tools | Mostly appropriate application | Partial application | Weak application |
| Solution Design | 25% | Solution is practical, scalable, and well-reasoned | Reasonable solution | Basic solution with gaps | Unclear solution |
| Security / Risk Awareness | 15% | Strong awareness of security and operational risks | Reasonable awareness | Limited awareness | Poor awareness |
| Clarity and Professionalism | 15% | Well-structured and professional response | Generally clear | Moderately clear | Poorly presented |

**Total Marks Awarded:**

## Question 1 — Retail Data Optimization Problem

**Problem Statement:** *A retail chain operates both physical stores and an online platform, generating large volumes of sales, inventory, and customer behaviour data. It wants real-time inventory tracking, demand forecasting, and personalized marketing recommendations. Design a big data architecture to support these requirements and explain the tools, technologies, and analytics models used.*

### 1. Understanding of Industry Context

The retail chain operates in an omnichannel environment: physical Point-of-Sale (POS) terminals, an e-commerce website/app, warehouses, and third-party logistics partners. Each channel independently generates high-velocity, high-volume, and high-variety data — structured transaction records, semi-structured clickstream/JSON logs, and unstructured customer reviews or support chats. This is a textbook Big Data problem exhibiting the classic 5 V's:

- **Volume** – millions of SKUs across thousands of stores and an online catalogue generate terabytes of transactional data daily.
- **Velocity** – POS scans, shelf-sensor pings, and website clicks arrive continuously and must be reflected in inventory counts within seconds.
- **Variety** – structured sales tables, semi-structured web logs, and unstructured customer feedback/social mentions.
- **Veracity** – inventory counts must reconcile across channels to avoid overselling or stockouts.
- **Value** – the ultimate business goal is converting this raw data into demand forecasts and personalized offers that drive revenue and reduce wastage.

### 2. Application of Big Data Concepts, Tools & Technologies

The architecture follows a modern Lambda-style design that separates a fast, real-time path (for inventory visibility) from a batch/analytical path (for forecasting and personalization), unified over a common storage layer:

- **Ingestion:** Apache Kafka (or AWS Kinesis) acts as the central nervous system, decoupling producers (POS, RFID/shelf-IoT sensors, mobile app, e-commerce

clickstream, ERP/supply-chain feeds) from consumers, and buffering bursts (e.g., festive-season sales spikes).

- **Stream Processing:** Apache Spark Structured Streaming or Apache Flink consumes the Kafka topics to update a real-time inventory view (store-level and warehouse-level stock) with sub-minute latency, and feeds low-latency dashboards for store managers.
- **Storage:** Raw and semi-structured data lands in a Data Lake (HDFS or Amazon S3) in its native format for cost-effective, schema-on-read storage; curated, query-ready data is loaded into a cloud Data Warehouse (Snowflake / Amazon Redshift / Google BigQuery) for BI and analytics.
- **Batch Processing:** Apache Spark (or Hadoop MapReduce for legacy batch jobs) performs nightly ETL — cleaning, deduplicating, and aggregating sales history for model training.
- **Analytics / ML models:** Time-series forecasting models such as ARIMA, Facebook Prophet, or LSTM neural networks predict SKU-level demand per store, factoring in seasonality and promotions. A recommendation engine (Spark MLlib collaborative filtering, or a deep learning two-tower model) generates personalized product recommendations from purchase history and browsing behaviour.
- **Serving & Visualization:** Power BI / Tableau dashboards for merchandising and marketing teams; a low-latency recommendation microservice (REST/gRPC API) that the e-commerce front-end and mobile app call in real time.

## 3. Proposed Architecture / Solution Diagram

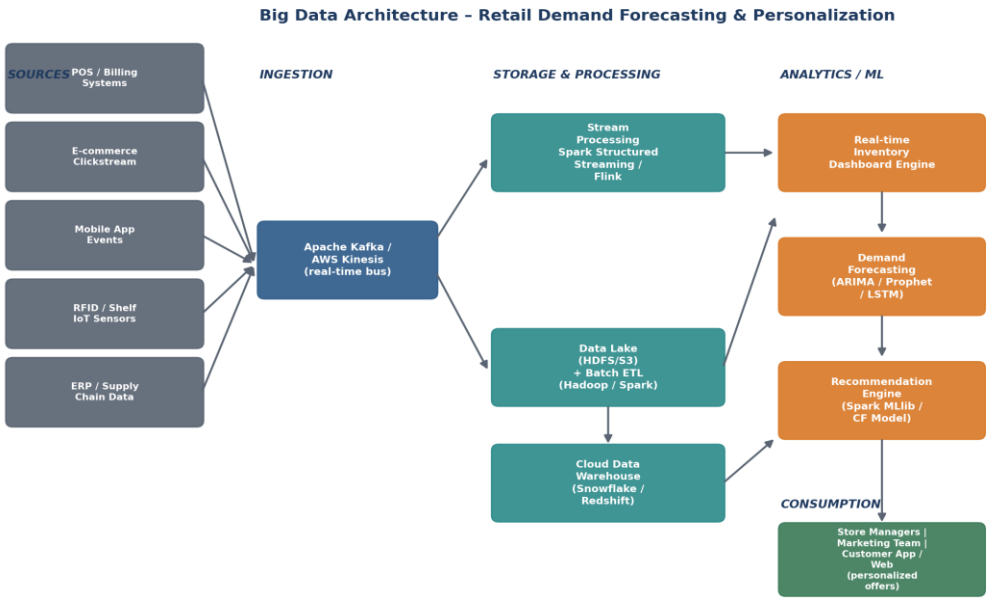


*Figure 1. End-to-end big data architecture for real-time inventory tracking, demand forecasting, and personalized marketing.*

## 4. Solution Design Rationale (Practicality & Scalability)

The design is deliberately layered and horizontally scalable: each layer (ingestion, stream processing, storage, analytics, serving) can scale independently as data volume grows, and Kafka provides natural fault-tolerance and replay capability if a downstream consumer fails. The dual storage strategy (data lake + warehouse) balances low-cost raw retention against fast, structured querying. Because the streaming and batch paths share the same Kafka source and data lake, the design avoids logic duplication and keeps forecasts consistent with the real-time inventory numbers shown to customers (e.g., “only 2 left in stock”).

## 5. Security & Risk Awareness

PII such as names, payment card data, and addresses must be tokenized/encrypted at rest (AES-256) and in transit (TLS). Role-Based Access Control (RBAC) restricts access to raw customer data; only aggregated, anonymized features are exposed to the recommendation model. Payment data handling must comply with PCI-DSS. Data retention and consent policies should align with regional regulations (e.g., India's DPDP Act / GDPR for international customers) especially for personalized marketing, which requires explicit customer consent (opt-in) and an easy opt-out mechanism.

**Question 2 — Social Media Fraud Detection Problem**
**Problem Statement:** *A social media platform processes billions of user interactions daily (posts, comments, likes, shares) and wants to detect fake accounts and misinformation quickly, with real-time or near-real-time processing. Design a scalable big data solution and explain the role of machine learning and stream processing frameworks.*

## 1. Understanding of Industry Context

This is an extreme-velocity, extreme-volume problem: billions of events per day means the platform must process on the order of tens of thousands of events per second at peak. Fake accounts and misinformation cause direct harm (financial scams, radicalization, erosion of trust) so detection latency directly affects the damage caused — this pushes the design toward near-real-time stream processing rather than end-of-day batch reports. The problem also has an adversarial dimension: bad actors actively adapt to evade detection, so the platform needs continuously retrained models, not a static rule set.

## 2. Application of Big Data Concepts, Tools & Technologies

A Lambda Architecture is well suited here, combining a speed layer for immediate flags and a batch layer for deep, historical model training:

- **Ingestion:** Apache Kafka partitions event streams (posts, comments, likes, shares, account-creation metadata) by topic/shard to sustain very high throughput with horizontal scaling.
- **Stream Processing (Speed Layer):** Apache Flink (preferred for true low-latency, stateful stream processing) or Spark Structured Streaming computes rolling features in real time — posting frequency, follower/following ratio, device fingerprint reuse, and content similarity — and stores them in a low-latency feature store (Redis) for millisecond lookups.
- **Machine Learning – fake account detection:** Since fake accounts often operate in coordinated networks (bot farms, follow-for-follow rings), a Graph Neural Network (GNN) or graph-analytics engine (e.g., Neo4j / GraphX on Spark) models the social graph and detects anomalous clustering and coordinated inauthentic behaviour that individual-account features would miss.
- **Machine Learning – misinformation detection:** NLP transformer models (BERT / RoBERTa fine-tuned for claim detection, or integration with fact-checking APIs)

classify post content and flag high-risk claims; multimodal models additionally examine images/video for manipulation.

- **Batch Layer:** Historical data in a Data Lake (HDFS/S3) is processed with Hadoop/Spark for large-scale model retraining, label curation from human-review outcomes, and trend/campaign analysis (detecting coordinated misinformation campaigns rather than one-off posts).
- **Action layer:** A rules + ML ensemble scoring engine decides to auto-flag, shadow-limit reach, or route to human moderators, balancing recall (catching bad actors) against precision (avoiding wrongful account suspension).

## 3. Proposed Architecture / Solution Diagram

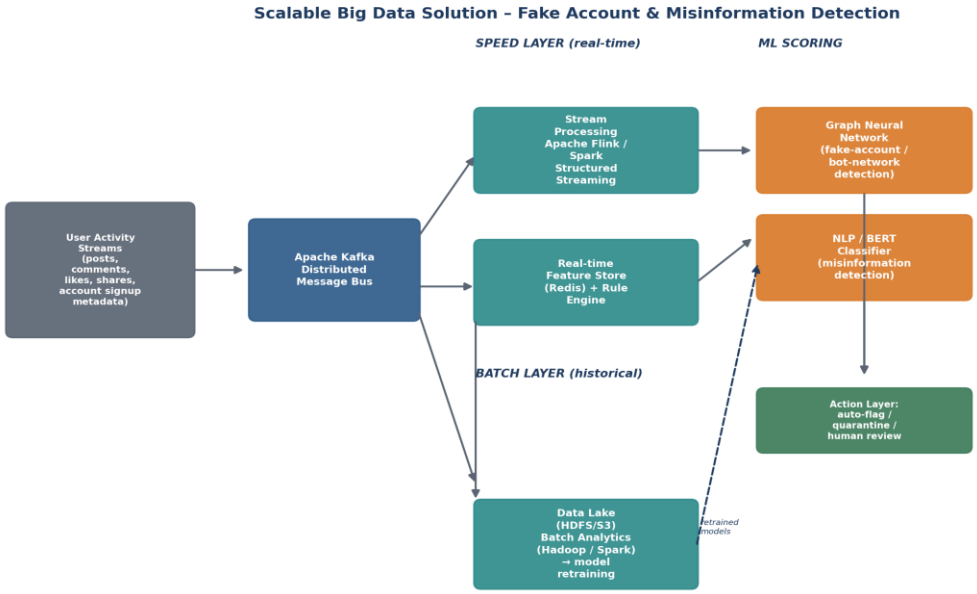


*Figure 2. Lambda-architecture solution combining real-time stream scoring with batch retraining for fake-account and misinformation detection.*

## 4. Solution Design Rationale (Practicality & Scalability)

The speed/batch split lets the platform act within seconds on obvious signals while continuously improving model accuracy from richer offline analysis. Kafka's partitioning and Flink's stateful, exactly-once processing guarantee the pipeline horizontally scales to billions of daily events without data loss. Keeping a human-in-the-loop review queue for borderline cases is a deliberate design choice to reduce false positives, which is critical given the reputational and legal risk of wrongly banning genuine users.

**5. Security & Risk Awareness**

Model explainability (e.g., SHAP values on the fraud/misinformation score) is required so moderators and appeals teams can justify actions; without it, the platform risks regulatory scrutiny and user backlash. Adversarial robustness testing is needed since fraudsters actively probe the system. User privacy must be protected — feature engineering should avoid over-collecting sensitive personal data, and access to raw behavioural logs should be restricted and audited under GDPR/data-protection law. Rate limiting and anomaly-based throttling protect the pipeline itself from being overwhelmed by coordinated attacks.

## Question 3 — Government Data Platform Problem

**Problem Statement:** *A government agency is building a national data platform combining census, economic, and geospatial datasets. Different departments need access to shared, integrated data; the system must ensure interoperability across agencies, and strict privacy/security requirements must be followed. Propose a governance and data management strategy.*

### 1. Understanding of Industry Context

Unlike the previous two problems, the core challenge here is not raw processing speed but data governance, interoperability, and trust across organizationally siloed agencies with different data standards, legacy systems, and legal mandates. Census data is highly sensitive (individual-level PII before aggregation), economic data is release-sensitive (market-moving if leaked early), and geospatial data (e.g., land records, infrastructure) has both operational and national-security implications. A successful platform must let departments share and combine data without duplicating it insecurely or losing track of provenance.

### 2. Application of Big Data Concepts, Tools & Technologies

The strategy centres on a zoned data lake plus a formal governance layer rather than a single tool:

- **Standardized ingestion:** Each agency publishes data through standardized REST/GraphQL APIs with mandatory metadata tagging (source, sensitivity classification, update frequency) at the point of entry, rather than ad-hoc file transfers.
- **Zoned Data Lake architecture:** A Raw Zone holds immutable, source-of-truth copies exactly as received (for audit/lineage); a Curated Zone holds cleaned, standardized, and mastered data (via Master Data Management, resolving e.g. the same citizen or the same geographic entity referenced differently by different agencies); a Consumption Zone exposes shared, ready-to-use datasets and APIs to authorized departments.
- **Common data models & open standards:** Adopting statistical exchange standards (SDMX) for census/economic data and OGC standards for geospatial data ensures true semantic interoperability, not just file-format compatibility.
- **Data Catalog & Lineage:** A searchable metadata catalog (e.g., Apache Atlas / DataHub) lets departments discover available datasets and trace exactly how a derived dataset was produced — essential for auditability in government use.

- **Data Governance Council:** A cross-agency body defines data ownership, quality standards, and approval workflows for new data-sharing agreements, supported by designated data stewards in each department.

## 3. Proposed Architecture / Solution Diagram

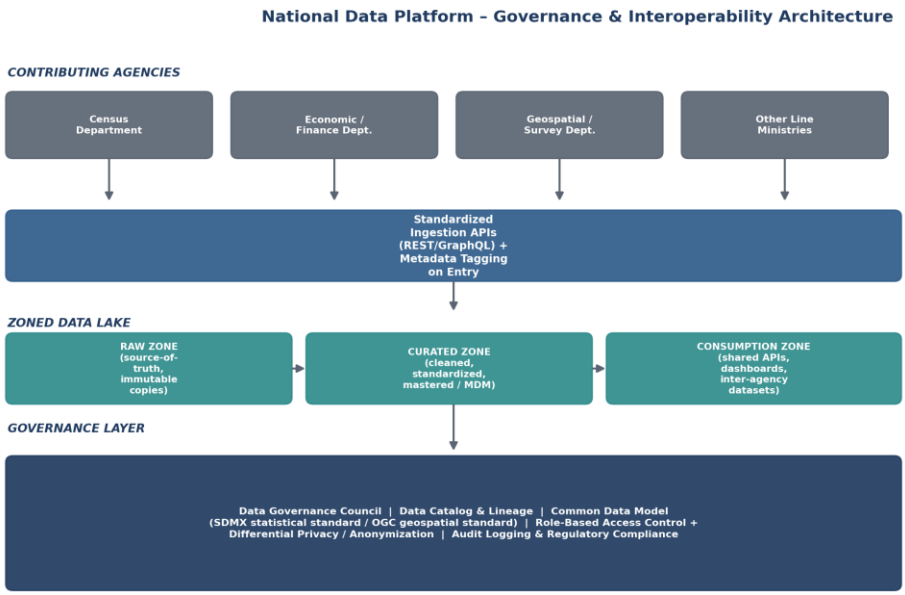


*Figure 3. Governance-first, zoned data lake architecture enabling secure, interoperable data sharing across government agencies.*

## 4. Solution Design Rationale (Practicality & Scalability)

The zoned lake design is practical and scalable because it lets each agency retain ownership and accountability for its raw data (Raw Zone) while still enabling controlled, standardized sharing (Curated/Consumption Zones) — avoiding the common failure mode of a "big bang" single shared database that no department trusts or fully controls. Using open interoperability standards (SDMX, OGC) instead of proprietary formats future-proofs the platform against vendor lock-in and eases onboarding of new departments.

## 5. Security & Risk Awareness

Because census microdata is highly sensitive, the platform applies Role-Based Access Control at the dataset and field level, combined with Differential Privacy or k-anonymity techniques when publishing aggregated statistics, so no individual can be re-identified. All access is logged for audit (who accessed what, when, and why), satisfying government audit and Right-to-Information requirements. Data classification tiers (public / restricted / confidential) drive automated encryption and access policies, and a formal data-sharing .

## Question 4 — Banking Fraud Detection Problem

**Problem Statement:** *A bank processes millions of financial transactions every minute and needs to detect fraudulent activities in real time, analyzing both historical and streaming transaction data, with low latency and high accuracy as critical requirements. Design a big data pipeline for fraud detection and explain the technologies and algorithms used.*

### 1. Understanding of Industry Context

Banking fraud detection is one of the most latency-critical big data use cases: a fraud decision must typically be returned in well under 100 milliseconds, at the moment of authorization, or the transaction simply proceeds. At the same time, false positives (blocking a legitimate customer) directly damage customer trust and generate costly manual review workload, so the system must jointly optimize for speed and accuracy while also using deep historical context (a customer's normal spending pattern) that a purely real-time system cannot compute from scratch on every transaction.

### 2. Application of Big Data Concepts, Tools & Technologies

A speed-layer/batch-layer (Lambda) pipeline, purpose-built for low-latency scoring, is appropriate:

- **Ingestion:** Apache Kafka ingests transaction events from card networks, ATMs, and mobile/net-banking channels with sub-millisecond producer-to-broker latency and guaranteed ordering per account.
- **Real-time feature computation:** Apache Flink (favoured over micro-batch frameworks for true event-at-a-time processing) computes rolling features — transaction velocity, deviation from typical spend, geolocation jump between consecutive transactions — and writes them to an in-memory feature store (Redis) for sub-millisecond retrieval during scoring.
- **Real-time ML scoring:** An ensemble of a supervised gradient-boosted model (XGBoost / Random Forest, trained on labelled historical fraud cases) and an unsupervised autoencoder (which flags transactions with high reconstruction error as anomalous, catching novel fraud patterns not seen in training data) is deployed via a low-latency model server (TensorFlow Serving / Triton Inference Server) to meet the sub-100ms SLA.

- **Decision engine:** Combines the ML score with hard business rules (e.g., hard transaction limits, blacklists) to approve, hold for step-up authentication, or block — rule-based checks act as a fast, interpretable safety net alongside the ML score.
- **Batch layer:** Historical transactions in a data warehouse/HDFS are processed with Spark/Hadoop for periodic model retraining, incorporating investigator feedback (confirmed fraud/false-positive labels) to reduce drift.

## 3. Proposed Architecture / Solution Diagram

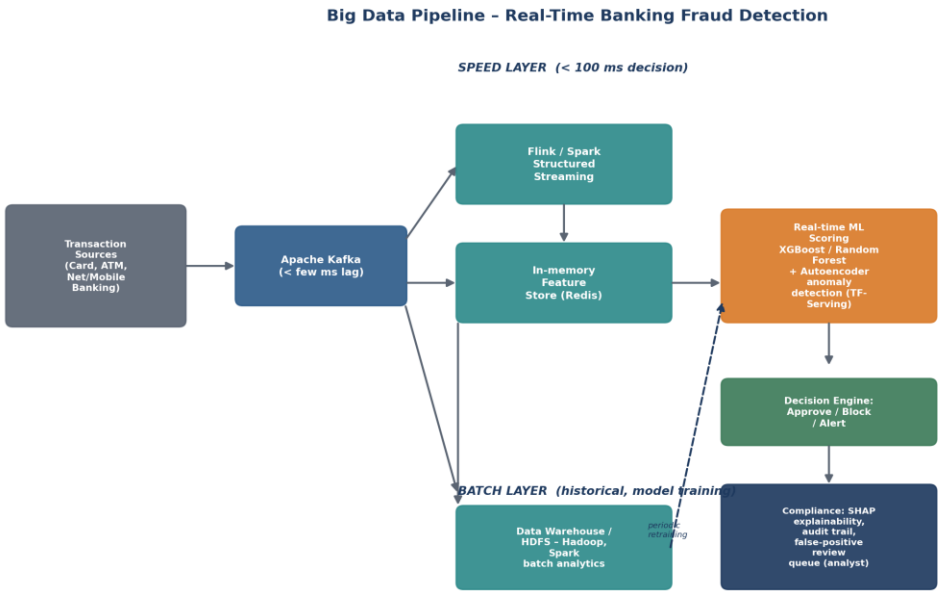


*Figure 4. Low-latency banking fraud detection pipeline combining stream-computed features, ensemble ML scoring, and rule-based decisioning.*

## 4. Solution Design Rationale (Practicality & Scalability)

Placing feature computation in an in-memory store and serving models through a dedicated low-latency inference server is the key design decision that makes sub-100ms decisions achievable at millions of transactions per minute — database lookups against a full historical warehouse at scoring time would be far too slow. The ensemble of supervised + unsupervised models balances known fraud patterns against emerging fraud types, improving accuracy beyond either model alone, while the periodic batch retraining loop keeps the system adaptive as fraud tactics evolve.

## 5. Security & Risk Awareness

End-to-end encryption (TLS in transit, AES-256 at rest) and tokenization of card/account numbers are mandatory, and the pipeline must comply with PCI-DSS. Because fraud/no-fraud decisions have direct financial and legal consequences, model explainability (SHAP values) is required so investigators and regulators can understand why a transaction was blocked, and a documented false-positive review/appeal process protects legitimate customers. Full audit trails of every scoring decision must be retained for regulatory examination, and models must be monitored for concept drift so accuracy does not silently degrade.

## Question 5 — Agriculture Smart Farming Problem

**Problem Statement:** *A smart farming system collects soil, weather, and crop data from sensors and drones providing real-time field information from multiple distributed sources. Farmers want insights to improve yield and resource usage. Design a big data solution for precision agriculture and explain how analytics can support decision-making.*

### 1. Understanding of Industry Context

Smart farming is fundamentally an edge-to-cloud IoT big data problem: data originates from many small, distributed, often connectivity-constrained sources (soil moisture/NPK sensors, weather stations, drone/satellite imagery, farm machinery telemetry) spread across large physical areas with unreliable rural internet. Unlike the banking or social-media cases, raw bandwidth and rural network reliability are real constraints, so processing some data at the edge before transmission is essential rather than optional, and the end-users (farmers) need simple, actionable insights rather than raw dashboards.

### 2. Application of Big Data Concepts, Tools & Technologies

The solution uses an edge-computing tier to pre-process and compress data before it reaches the cloud analytics layer:

- **Edge/Field layer:** IoT soil-moisture and weather sensors, drone/satellite imagery, and farm-machinery/irrigation sensors generate continuous but geographically distributed readings.
- **Edge gateway:** A local gateway performs pre-processing (filtering noise, basic aggregation, image compression) and communicates using the lightweight MQTT protocol, which is designed for low-bandwidth, high-latency, intermittent rural connectivity — far more efficient than raw HTTP for thousands of sensor pings.
- **Cloud ingestion & storage:** Kafka ingests the pre-processed data into a Data Lake (HDFS/S3) for long-term storage of raw readings and imagery.
- **Real-time processing:** Spark Structured Streaming generates time-critical alerts (e.g., soil moisture dropping below a crop-specific threshold triggers an irrigation alert) with minimal delay.
- **Analytics / ML models:** Regression and machine-learning models (Random Forest / gradient boosting) predict crop yield from combined soil, weather, and historical harvest data; Convolutional Neural Networks (CNNs) analyze drone/satellite imagery

to detect early signs of crop disease, pest stress, or nutrient deficiency by field zone, well before it is visible to the naked eye.

- **Decision support:** A simple farmer-facing mobile app/dashboard translates model outputs into concrete recommendations — irrigation scheduling, fertilizer dosage by zone (precision/variable-rate application), and pest-control timing — rather than exposing raw sensor data.

## 3. Proposed Architecture / Solution Diagram

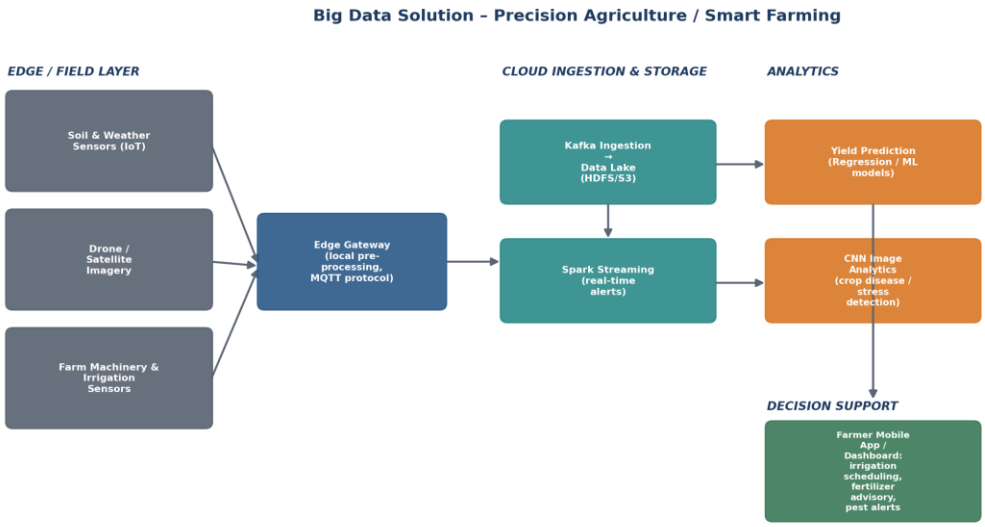


*Figure 5. Edge-to-cloud precision agriculture architecture, from distributed field sensors to farmer-facing decision support.*

## 4. Solution Design Rationale (Practicality & Scalability)

Introducing an edge-computing tier is the central design decision that makes this solution practical under real rural-connectivity constraints — it reduces the volume of data transmitted, lowers cloud ingestion cost, and still enables near-real-time irrigation alerts even during intermittent connectivity, since urgent thresholds can be evaluated locally at the gateway before cloud sync. Combining structured sensor data (regression models) with unstructured drone imagery (CNNs) gives a more complete, zone-level picture of field health than either data type alone, directly supporting the farmer's goal of improving yield and resource efficiency.

## 5. Security & Risk Awareness

Field IoT devices must use mutual device authentication and encrypted MQTT (MQTTS/TLS) to prevent spoofed sensor readings that could mislead irrigation or fertilizer decisions. The system should be designed for graceful degradation under connectivity loss (local buffering and later sync) rather than data loss. Data ownership is a key trust issue for farmers: policies should make clear that farm-level data belongs to the farmer/cooperative, with explicit consent required before it is aggregated, resold, or used to train models shared across other farms.